
2) Solved Lab Activities (Allocated Time 1 Hour)

Activity 1:

The BFS search starts from a node, then it checks all the nodes at distance one from the beginning node, then it checks all the nodes at distance two, and so on. So as to recollect the nodes to be visited, BFS uses a queue.

The steps of the algorithm work as follow:

1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

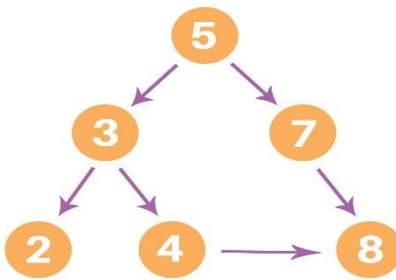


Figure: Graph for BFS Implementation

Many times, a graph may contain two different disconnected parts and therefore to make sure that we have visited every vertex, we can also run the BFS algorithm at every node.

```
graph = {  
    '5' : ['3', '7'],  
    '3' : ['2', '4'],  
    '7' : ['8'],  
    '2' : [],  
    '4' : ['8'],  
    '8' : []  
}
```

##The above code will create a graph with the entries given in the figure above.

```

visited = [] # List for visited nodes.
queue = []   #Initialize a queue

def bfs(visited, graph, node): #function for BFS
    visited.append(node)
    queue.append(node)

    while queue:          # Creating loop to visit each node
        m = queue.pop(0)
        print (m, end = " ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, '5')    # function calling

```

Activity 2:

Try implementing BFS for a tree now, using the following tree structure:

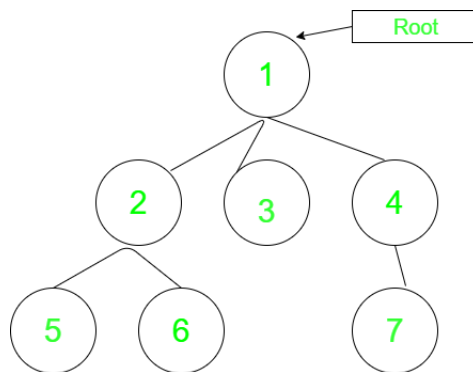


Figure: Tree for BFS Implementation

The code to make the tree is given as

```

# python program to demonstrate some of the above
# terminologies

# Function to add an edge between vertices x and y

# Function to print the parent of each node

```

```
def printParents(node, adj, parent):

    # current node is Root, thus, has no parent
    if (parent == 0):
        print(node, "->Root")
    else:
        print(node, "->", parent)

    # Using DFS
    for cur in adj[node]:
        if (cur != parent):
            printParents(cur, adj, node)

# Function to print the children of each node
def printChildren(Root, adj):

    # Queue for the BFS
    q = []

    # pushing the root
    q.append(Root)

    # visit array to keep track of nodes that have been
    # visited
    vis = [0]*len(adj)

    # BFS
    while (len(q) > 0):
        node = q[0]
        q.pop(0)
        vis[node] = 1
        print(node, "-> ", end=" ")

        for cur in adj[node]:
            if (vis[cur] == 0):
                print(cur, " ", end=" ")
                q.append(cur)
        print("\n")

# Function to print the leaf nodes
def printLeafNodes(Root, adj):
```

```
# Leaf nodes have only one edge and are not the root
for i in range(0, len(adj)):
    if (len(adj[i]) == 1 and i != Root):
        print(i, end=" ")
print("\n")

# Function to print the degrees of each node
def printDegrees(Root, adj):

    for i in range(1, len(adj)):
        print(i, ": ", end=" ")

        # Root has no parent, thus, its degree is equal to
        # the edges it is connected to
        if (i == Root):
            print(len(adj[i]))
        else:
            print(len(adj[i])-1)

# Driver code

# Number of nodes
N = 7
Root = 1

# Adjacency list to store the tree
adj = []
for i in range(0, N+1):
    adj.append([])

# Creating the tree
adj[1].append(2)
adj[2].append(1)

adj[1].append(3)
adj[3].append(1)

adj[1].append(4)
adj[4].append(1)

adj[2].append(5)
adj[5].append(2)
```

```
adj[2].append(6)
adj[6].append(2)

adj[4].append(7)
adj[7].append(4)

# Printing the parents of each node
print("The parents of each node are:")
printParents(Root, adj, 0)

# Printing the children of each node
print("The children of each node are:")
printChildren(Root, adj)

# Printing the leaf nodes in the tree
print("The leaf nodes of the tree are:")
printLeafNodes(Root, adj)

# Printing the degrees of each node
print("The degrees of each node are:")
printDegrees(Root, adj)
```

Activity 3:

Using the code in ‘activity 1’ apply the same procedure to the graph give in the Figure below:

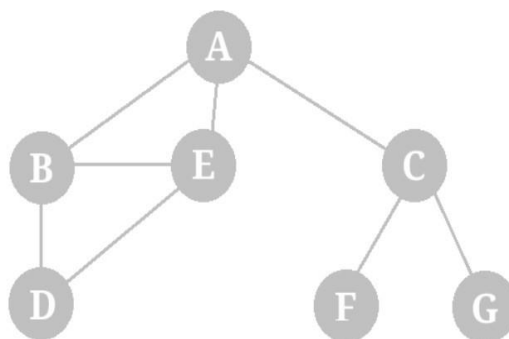


Figure: Graph for Activity 3

Activity 4:

Try changing the start and goal states for the Activity 3 implementation and observe what happens.

3) Graded Lab Tasks (Allotted Time 1.5 Hour)

Lab Task 1:

Imagine the same graph we discussed above but this time we also mention the cost of each edge.

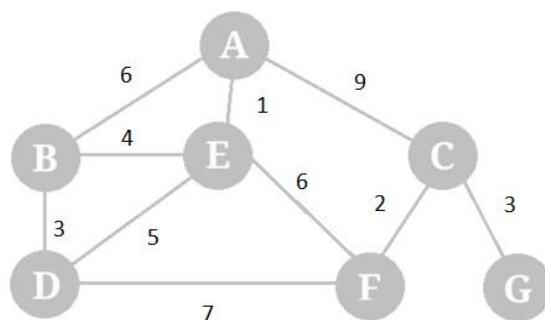


Figure: Weighted Graph for Activity Lab Task 1

First we modify our graph structure so that in Node.actions is an array of tuples where each tuple contains a vertex and its associated weight.

```
graph = { 'A': Node('A', None, [('B', 6), ('C', 9), ('E', 1)], 0),
          'B': Node('B', None, [('A', 6), ('D', 3), ('E', 4)], 0),
          'C': Node('C', None, [('A', 9), ('F', 2), ('G', 3)], 0),
          'D': Node('D', None, [('B', 3), ('E', 5), ('F', 7)], 0),
          'E': Node('E', None, [('A', 1), ('B', 4), ('D', 5), ('F', 6)], 0),
          'F': Node('F', None, [('C', 2), ('E', 6), ('D', 7)], 0),
          'G': Node('G', None, [('C', 3)], 0) }
```

Once the graph is updated, try applying BFS algorithm and calculate the total cost from start to goal state (A to G).

Try different variations of start and goal states and see what happens to path costs.

Lab Task 2:

In order to move onto more complex implementations of BFS let us consider a concise version of Travelling Salesman Problem given in Figure below:

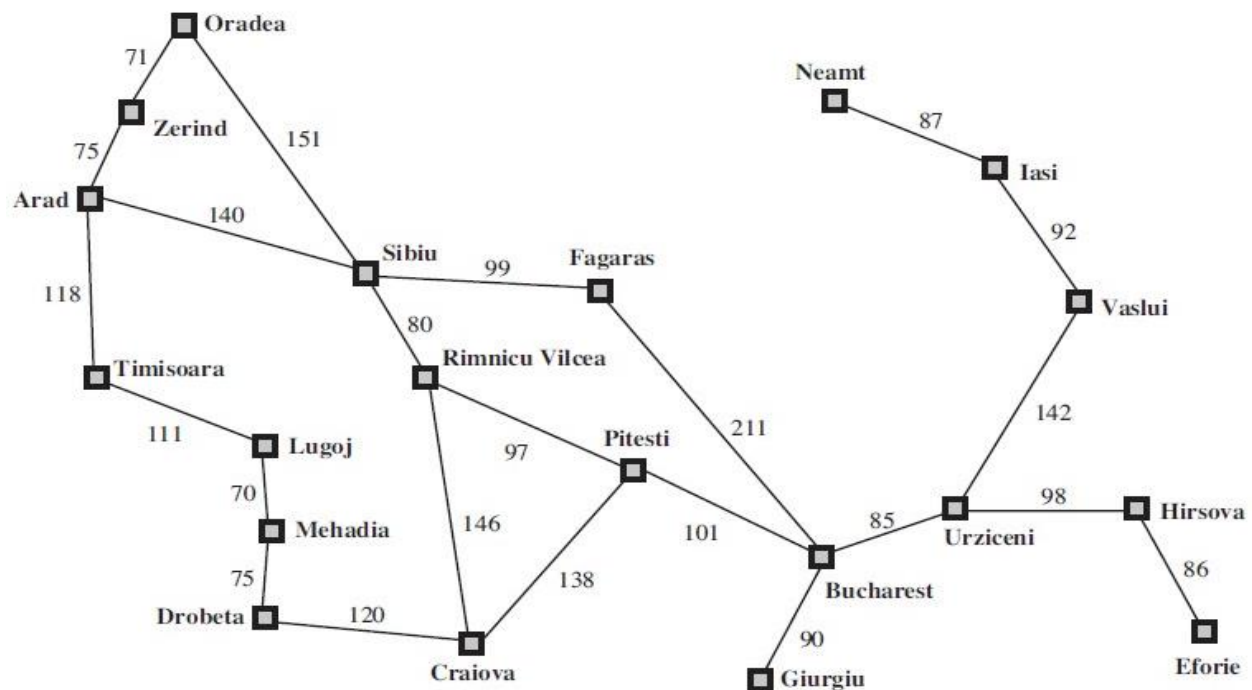


Figure: Travelling Salesman Problem

- Draw a corresponding graph from the above Figure from Arad to Bucharest containing all possible nodes. (Arad as Start Node and Bucharest as Goal Node)
- Apply breadth first search to reach the goal state, your output should have the path and the total distance to reach the path.
- Save your code and results as BFS_REG#.py as it would be used in later exercises.